

Arrays

- objective

- Understand what is an array ✓
- Learn to declare and initialize array.
- Understand need of arrays. ✓

⇒ Why do we need arrays?

Problem I: Input - marks of 30 students for 1 subject.
Output - average mark.

Solution: Is array needed?

No array
needed.

```
int sum=0, marks;  
float average;  
for(i=0; i<30; i++) {  
    scanf("%d", &marks);  
    sum += marks;  
}  
average = sum/30.0;
```

Problem II: Input - marks of 30 students for 1 subject.
Output - deviation of each value from average

I/P

12	- 15 =	-3
16	- 15 =	1
18	- 15 =	3
14	- 15 =	-1

Solution: Marks of students need to be stored in memory.
So, we need array.

→ Array can store multiple values simultaneously.

⇒ what is an array?

Example

int num = 10, arr[3];

0 to 2

Assumption
int = 4 byte

108	?	arr[2]
104	37	arr[1]
100	}	arr[0]
	100	num

another example

double xyz[10];

Declaration

num = 100; ✓

arr[1] = 37; ✓

arr = 37; ✗ Invalid - no index

arr[3] = 37; ✗ Invalid - index out of range

int pqr = arr[1]; ✓

→ A sequenced collection of related data items that share a common name.

→ same type

arr[2] = arr[1] + 7; // No change in value of arr[1].

arr[2] = arr[0+1]; ✓

↑
integer ✓

✓ arr[j*2] = arr[i+3]; // i and j are int

└──┬──┘
Should be valid indices.

printf("%d", arr[i]);

scanf("%d", &arr[i]);

int marks[50];

└─┬─┘
0 to 49

marks[-1] = 99; ✗
marks[50] = 33; ✗

marks[77] = 66; ✗
marks[-10] = 45; ✗
marks[10.3] = 7; ✗

⇒ Program to take 5 float nums, and print in reverse.

```
#include <stdio.h>
```

```
int main() {
```

```
    float numbers[5];
```

```
    for(int i=0; i<5; i++)
```

```
        scanf("%f", &numbers[i]);
```

```
    for(int i=4; i>=0; i--)
```

```
        printf("%f", numbers[i]);
```

```
    return 0;
```

```
}
```

	i
9.3	numbers[4]
8.1	numbers[3]
2.0	numbers[2]
6.2	numbers[1]
3.7	numbers[0]

Que:1

```
int arr[5];
```

```
arr[5] = 3; X
```

✓

6

val

Que:2

```
int arr[3], val = 1;
```

```
arr[val+1] = val + 5;
```

```
val = arr[val+1];
```

6

arr[2]

?

arr[1]

?

arr[0]

Que:3

```
float arr[3], val = 1.1;
```

```
arr[val+1] = val + 5;
```

```
val = arr[val+1];
```

Invalid

Assignment

→ Take n from input

→ Take mark of n students from input.

→ Output deviation of marks of each students from average.

- ⇒ How to initialize an array?
- compile-time initialization
 - run-time initialization.

→ compile-time initialization

`int arr[3] = {7, 12, 23};`

`int arr[];` X

23	arr[2]
12	arr[1]
7	arr[0]

✓✓ `int arr[] = {8, 18, 12, 4, 6};`

Size? 5

6	arr[4]
4	.
12	.
12	.
8	arr[0]

✓ `float farr[5] = {1.2, 3.3};`

→ partial compile time initialization of array leaves other elements as 0.

0	farr[4]
0	.
0	.
3.3	.
1.2	farr[0]

Common requirement

`int arr[100000] = {0};`
 ↳ All int 0.

`int arr[3] = {1, 2, 6, 9, 2};` X Not valid

→ some compiler does not raise error.

→ Run-time initialization

run-time
initialization

```
int marks[5];  
for(int i=0; i<5; i++)  
    marks[i] = i+30;  
    // i*5+30;  
    // i
```

34	marks[4]
33	.
32	.
31	.
30	marks[0]

⇓

```
for (int i=0; i<5; i++)  
    scanf("%d", &marks[i]);
```